



RWA NOVA(React + Next.js)

1. 프로젝트 개요

- **프로젝트명:** RWA NOVA(RWA 토큰 기반의 금융 플랫폼)
- **개발 기간:** 2025.01 ~ 2025.03, 2025.05 ~ 2025.06
- **수행 방식:** 팀 협업 / Git 기반 협업
- **담당 범위:** RWA NOVA 사용자 프론트엔드 개발
- **배포 현황:** 실제 서비스용으로 개발되었으며, 현재 서비스 운영 중
- **기술 스택:** Next.js, React, TypeScript, Sass, Framer Motion, GSAP, Apexchart, Reown, Ethers.js

2. 클라이언트 요구사항 및 나의 역할

클라이언트 개요

RWA(Real-World Asset)를 토큰화하여 부동산, 채권 등 실물 자산을 디지털 자산으로 전환하고, 이를 거래, 투자, 스테이킹할 수 있는 블록체인 기반의 금융 플랫폼

요구사항 요약

- **사용자 경험(UX) 극대화:** 페이지 인터랙션 및 애니메이션 효과를 활용하여 몰입감 있는 사용자 경험을 제공.
- **마이페이지 기능:** 스테이킹, P2P 거래, 자산 관리 등 다양한 메뉴를 포함한 마이페이지 기능 구현 필요.
- **유연한 UI 시스템:** 기능 및 디자인이 다양한 테이블 컴포넌트를 재사용성 높게 구현.
- **SEO 최적화:** 기본적인 메타데이터뿐만 아니라 Schema Markup 등 고급 SEO 기술을 적용.

- **복잡한 회원 인증 처리:** reown을 기반으로 모달을 통한 회원가입 및 로그인 예외 처리 로직이 필요.

담당 역할

- **인터랙티브 UI/UX 개발:** 스크롤 트리거, 커서 인터랙션, 페이드인/아웃, 배경 애니메이션 등 다양한 인터랙션 및 애니메이션 효과를 GSAP와 Framer Motion을 활용해 구현.
- **유저 인증 및 예외 처리:** reown을 사용한 모달 기반 회원가입 및 로그인 기능 개발과 복잡한 예외 케이스(실패, 오류 등)에 대한 UI/UX 처리.
- **다기능 테이블 컴포넌트 개발:** 스테이킹, P2P, 마이페이지 등 여러 페이지에 사용되는 다양한 테이블 UI를 컴포넌트화하여 구현, 각 테이블마다 다른 기능과 UI를 유연하게 적용.
- **SEO 최적화:** 페이지별 메타데이터 설정은 물론, SEO를 강화하기 위해 Schema Markup을 직접 작성하고 적용.

3. 기술 스택 및 아키텍처

항목	사용 기술 / 라이브러리
프레임워크	Next.js, React, TypeScript
상태 관리	React Context API
UI/CSS	Sass(SCSS), Framer Motion, GSAP, Apexchart
API 통신	Axios
블록체인 연동	Reown, Ethers.js
SEO	Next.js Metadata, Schema Markup
협업 도구	BitBucket

아키텍처 특징

- **동적 페이지 및 라우팅:** Next.js를 사용해 MPA와 SPA의 장점을 결합한 하이브리드 구조를 구축하고, 동적 라우팅을 통해 페이지를 효율적으로 관리. 주요 페이지는 각각의 폴더 내 `page` (SEO·라우팅)와 `page-content` (UI·비즈니스 로직)로 분리.
- **Context API 기반 상태 관리:** `React Context API` 를 활용하여 전역 상태(예: 인증 상태, 지갑 정보)를 관리하고, 컴포넌트 간의 데이터 흐름을 체계적으로 구성.
- **API 사용:** `axios` 를 기반으로 API 요청을 처리하는 커스텀 훅(`use-fetch-base.ts`)을 만들어 API 통신 로직을 추상. 각 기능별로 API 서비스를 별도의 파일로 분리하여 코드의 가독성과 유지보수성을 높임.

4. 주요 기능

기능	설명
마켓플레이스	실물 자산(RWA)을 토큰화한 상품들의 목록을 제공하며, 각 상품의 상태, 가격, APR(Annual Percentage Rate) 등 상세 정보를 확인하고 투자하는 기능.
P2P거래	사용자 간에 토큰화된 자산을 직접 거래할 수 있는 기능. 오더북, 거래 체결 내역 등 실시간 데이터를 제공.
스테이킹	사용자가 보유한 토큰을 예치하여 보상을 받을 수 있는 스테이킹 풀 관리 기능. 풀 생성, APR, 수수료 등의 정보를 제공하고 입/출금 처리.
마이페이지	사용자별 자산 현황, 거래 내역, 스테이킹 현황 등을 한눈에 확인할 수 있는 개인 대시보드.
관리자 페이지	상품, 공지사항, FAQ 등 콘텐츠 관리 및 사용자 관리 기능.

5. 문제 해결 및 기술적 도전

✓ 문제 1: 복잡한 인증 로직 및 모달 컴포넌트 관리

페이지 이동 없이 모달을 통해서만 회원가입 및 로그인을 처리해야 했으며, 이 과정에서 발생하는 다양한 예외 케이스(로그인 실패, 사용자 미발견, 유효성 검사 등)를 유기적으로 처리.

→ `reown`을 활용한 월렛 연결 및 서명 요청부터 로그인/회원가입 분기 처리까지, 전체 인증 플로우를 React Hooks와 Context API로 직접 설계. `SignupModal`에서는 자체적으로 상태(`useState`)와 유효성 검사 로직을 구현하여 사용자 입력 및 약관 동의 상태를 관리했고, 모달 컴포넌트를 이메일 인증, 가입 완료 등 단계별로 세분화하여 복잡한 플로우를 모듈화. 모달 컴포넌트를 여러 개로 분리하여 각 상태에 따른 UI를 명확히 처리, 복잡한 인증 과정을 안정적으로 구현하고 관리하는 구조를 마련.

✓ 문제 2: 동적 필터링, 검색, 다양한 기능 구현

마켓플레이스, 스테이킹 등 여러 페이지에서 필터와 검색 조건에 따라 데이터를 동적으로 업데이트해야 했으며, 필터링 시 페이지 초기화, 검색어 입력 후 특정 액션(엔터, 아이콘 클릭) 시에만 결과를 반영하는 디테일한 로직 구현이 필요. 이 외에도 문서 다운로드, 에디터 내용 표시, reCAPTCHA 등 다양한 기능 구현이 요구.

→ 검색어와 최종 검색어 상태를 분리하고, 엔터키나 검색 아이콘 클릭 시에만 최종 검색어를 업데이트하도록 설계. 필터나 최종 검색어가 변경될 때마다 페이지를 초기화하고 데이터

를 다시 불러오는 로직을 구현. `FilterButtonGroup` 컴포넌트를 만들어 여러 필터를 중복 선택하고 해제할 수 있는 유연한 필터링 시스템을 구축. 또한 문서 다운로드, 텍스트 에디터(ql-editor) 내용 표시, 보안 강화를 위한 reCAPTCHA 등 여러 부가 기능들을 개발.

✓ 문제 3: 다양한 테이블 컴포넌트의 비효율성

스테이킹, P2P, 마이페이지 등 여러 페이지에 걸쳐 다양한 기능과 UI를 가진 테이블이 존재했고, 이를 개별적으로 구현할 경우 코드 중복과 유지보수 문제 발생이 예상.

→ 범용적인 `DataTable` 컴포넌트를 만들고, `StakingDataTable` 처럼 사용 목적에 맞는 맞춤형 컴포넌트를 별도로 제작. 테이블 헤더 옵션을 JSON 형식으로 구성하여 Props로 전달받아 동적으로 테이블을 구성하는 구조를 설계, 코드 재사용성과 유지보수성을 극대화.

✓ 문제 4: 페이지 인터랙션 및 애니메이션 성능 최적화

스크롤 트리거, 카운팅, Parallax 등 다양한 애니메이션 효과를 다수 적용하면서 웹 성능 저하가 우려.

→ `react-intersection-observer`를 활용한 커스텀 훅을 직접 만들어, 사용자가 특정 섹션에 진입했을 때만 애니메이션이 실행되도록 구현. 또한 `GSAP`와 `Framer Motion`을 도입해 애니메이션 로직을 컴포넌트와 분리하고, 불필요한 리렌더링을 최소화. 특히 메인 페이지의 카운팅 애니메이션은 `IntersectionObserver`로 가시성을 확인하고, `animateValue` 헬퍼 함수를 통해 직접 구현하여 성능을 최적화하고 부드러운 사용자 경험을 유지.

✓ 문제 5: 동적 페이지의 SEO 최적화

마켓플레이스 상세 페이지와 같은 동적 페이지의 경우, 정적인 정보만으로는 효과적인 SEO를 달성하기 어려웠고, 페이지별로 고유한 콘텐츠에 맞는 메타데이터와 Schema Markup을 동적으로 생성할 필요가 있었음.

→ Next.js의 `generateMetadata` 함수를 활용해 서버에서 API 통신을 통해 페이지의 제목, 설명, OG(Open Graph) 이미지 등 고유 데이터를 받아와 동적으로 메타데이터를 생성. 클라이언트 측에서는 `useEffect`를 사용해 API로 불러온 데이터를 기반으로 `schemaMarkup` JSON-LD를 동적으로 생성, `document.head`에 삽입하여 검색 엔진이 콘텐츠의 구조를 정확하게 파악하도록 구현.

6. 협업 및 커뮤니케이션

- Bitbucket 기반 Git 협업과 코드 품질 개선 및 지식 공유.
 - 기획이 변경되는 상황이었기 때문에, 기획 및 디자인 담당자와 지속적으로 소통. 동료 개발자가 담당할 관리자 프론트와 사용자 프론트 간 발생하는 문제에 대해 함께 고민하고 해결 방안을 모색.
-

7. 개발 결과 및 회고

결과

- RWA 토큰의 거래, 스테이킹, P2P 거래를 지원하는 사용자 친화적인 웹 플랫폼을 성공적으로 구축.
- 인터랙티브한 페이지 디자인과 애니메이션을 통해 사용자 참여도를 높이고, 시각적으로 매력적인 서비스를 제공.
- SEO를 위한 기술적 최적화를 통해 검색 엔진 노출도를 향상시키고, 잠재 사용자 유입의 기반 마련.

회고

- 복잡한 블록체인 기반의 금융 서비스(스테이킹, P2P)를 구현하며 다양한 UI/UX 케이스를 처리하는 경험.
 - 테이블 컴포넌트를 범용적으로 설계하며 재사용성에 대해 깊이 고민할 수 있었던 것이 인상 깊음.
 - 동적인 인터랙션과 애니메이션을 효과적으로 구현하는 과정에서 성능 최적화의 중요성을 인식함.
-

8. 스크린샷

- 앱 주요 화면별 캡처 이미지 (홈 / 기능 A / 기능 B 등)

RWA Fractional Investment

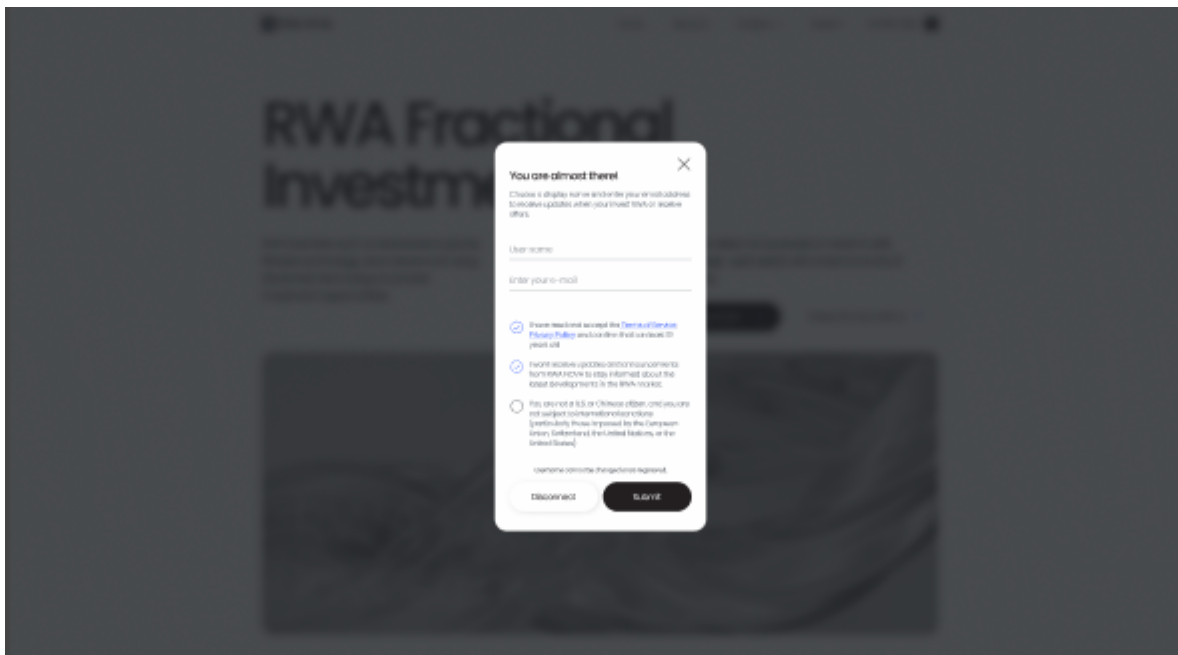
RWA tokenizes such as real estate & spaces lifestyle, technology, and culture & art using blockchain technology to provide investment opportunities.

This enables many people to invest in safe and high-yield assets with small amounts of money.

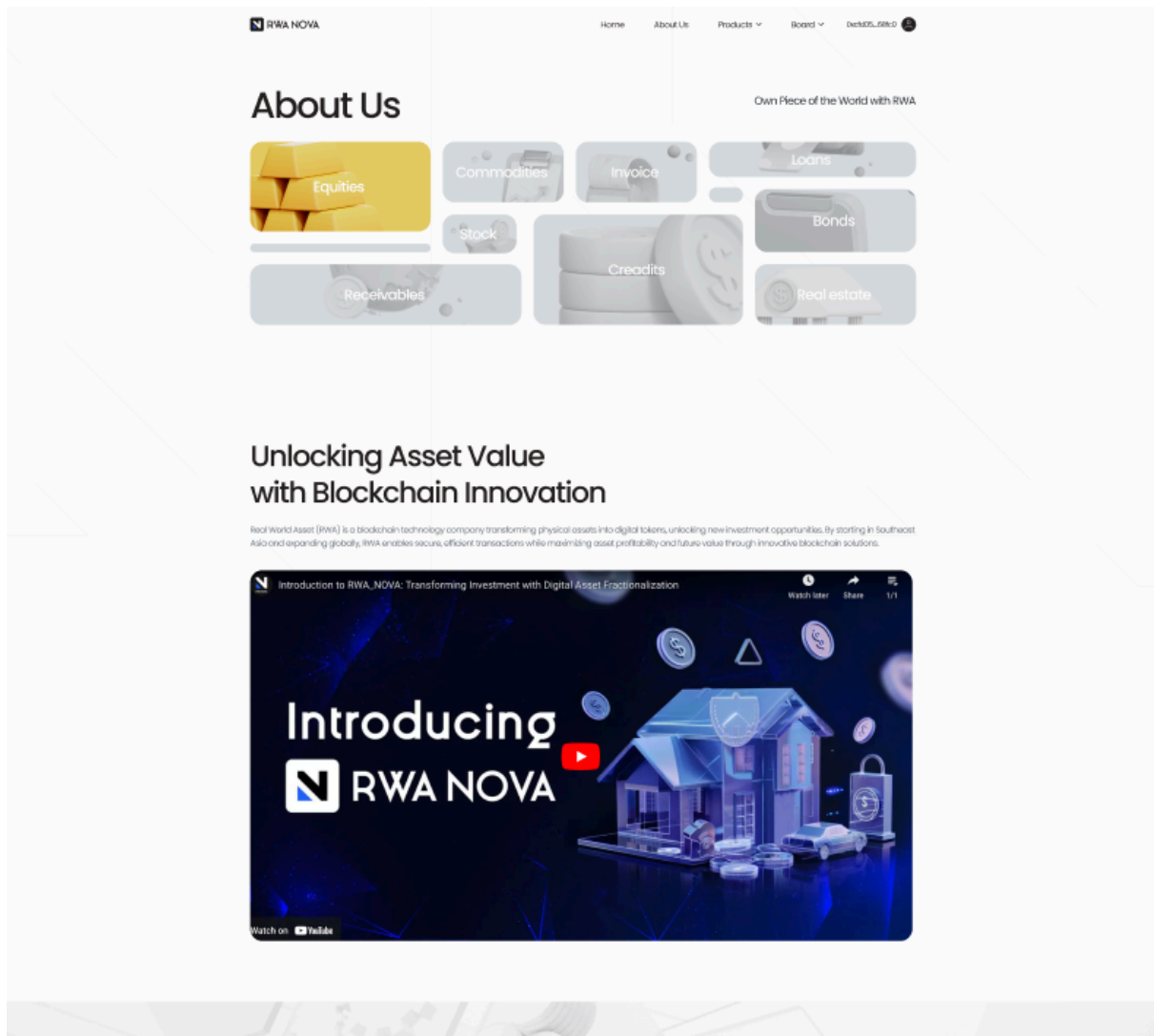
[How to join](#)[Shape the Future with Us](#)

RWA Fractional Investment

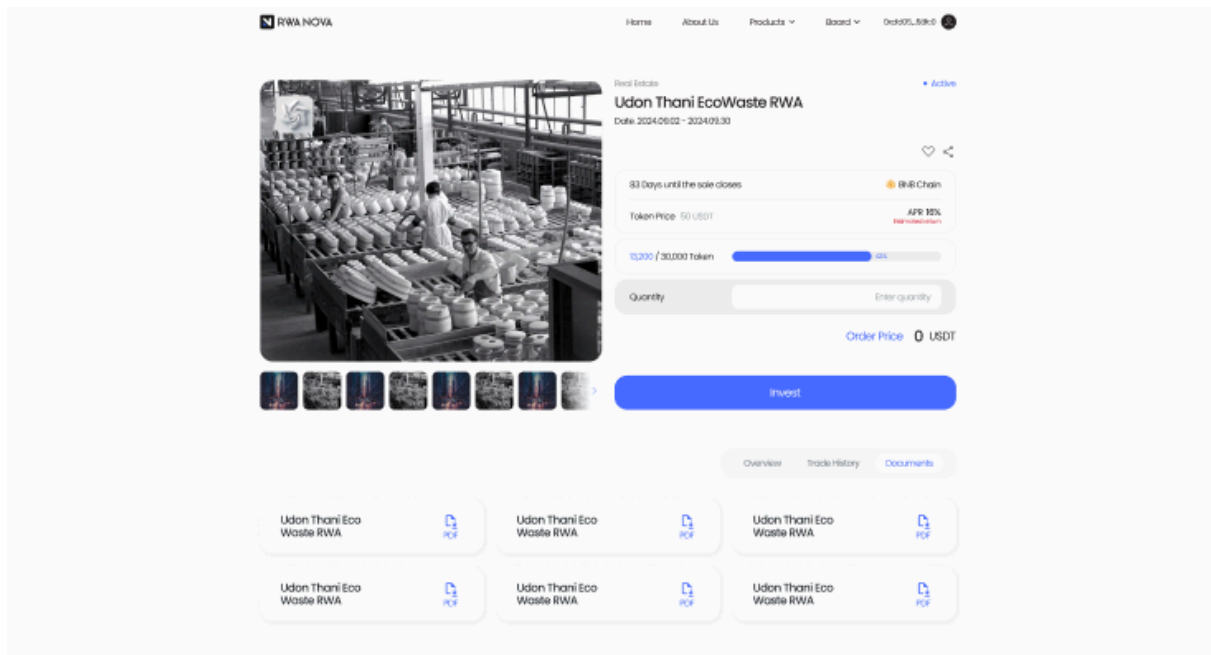
이미지 1. 메인 페이지



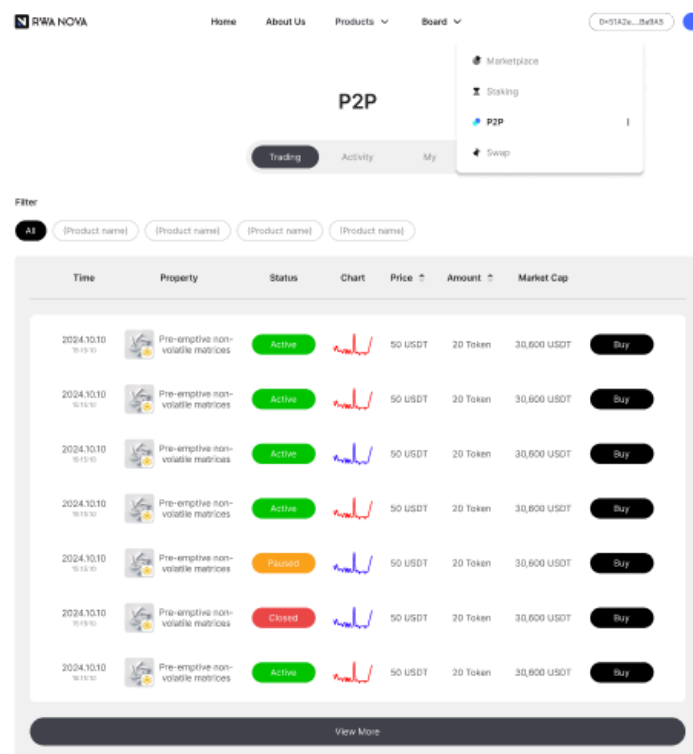
이미지 2. 회원가입 모달



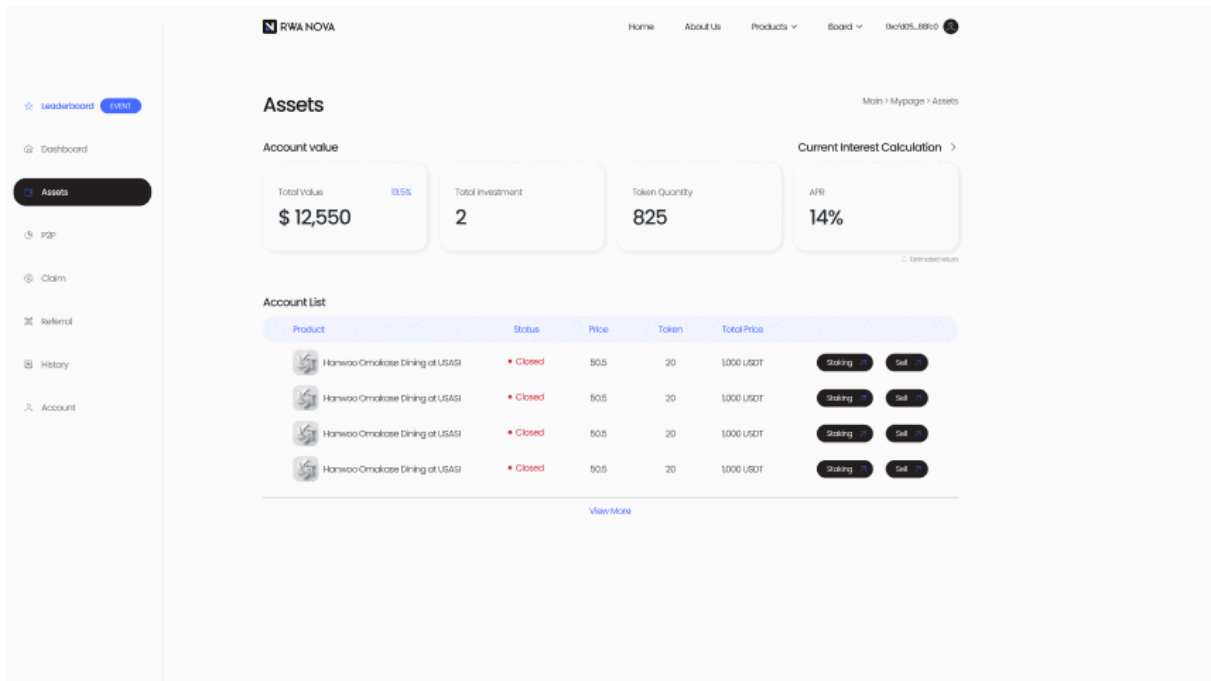
이미지 3. 어바웃 페이지



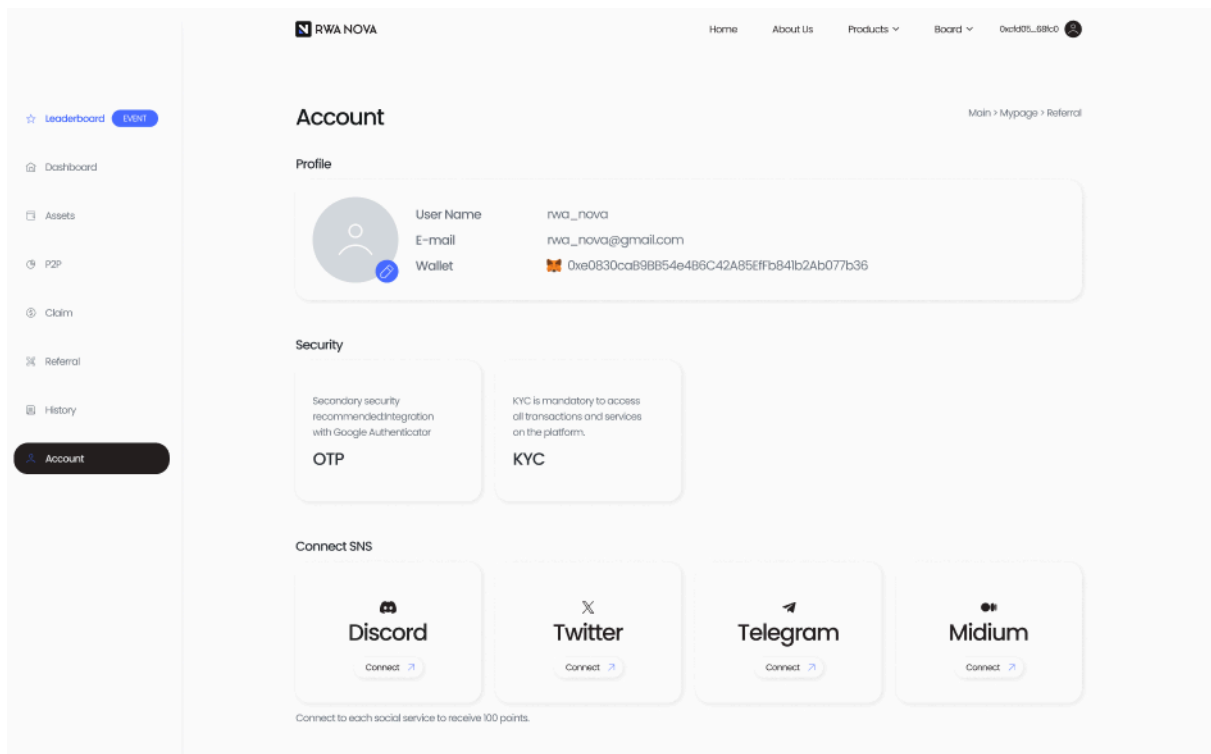
이미지 4. 마켓플레이스 상세 페이지



이미지 5. P2P 페이지



이미지 6. 마이페이지 에셋 페이지



이미지 7. 마이페이지 어카운트 페이지

9. 표지 이미지

RWA Fractional Investment

RWA tokenizes such as real estate & spaces lifestyle, technology, and culture & art using blockchain technology to provide investment opportunities.

This enables many people to invest in safe and high-yield assets with small amounts of money.

How to join [↗](#)

Shape the Future with Us [↗](#)

RWA Fractional Investment

»